DistributedPanGA: distributed pangenome self-alignment that reproduces FastGA

Saksham Diwan[1], Harshaan Chugh[1], Shuo Cao[2], Erik Garrison[2*], Giulia Guidi[1,3*]

[1*]Department of Computer Science, Cornell University, 107 Hoy Rd, Ithaca, 14853, New York, USA.

[2*]Department of Genetics, Genomics and Informatics, University of Tennessee Health Science Center, Memphis, 38163, Tennessee, USA.

[3]Applied Mathematics and Computational Research Division, Lawrence Berkeley National Laboratory, 1 Cyclotron Road, Berkeley, 94720, California, USA.

*Corresponding author(s). E-mail(s): egarris5@uthsc.edu; gg434@cornell.edu;
Contributing authors: sd966@cornell.edu; hsc53@cornell.edu;

Abstract

Motivation: Self-aligning a pangenome of $\boldsymbol{H}$ haplotypes with FastGA finds matches across all haplotype pairs in a single scan of one sorted $\boldsymbol{k}$-mer table, so the scan grows as $\boldsymbol{O(H \cdot L)}$ rather than the $\boldsymbol{O(H^2 \cdot L)}$ of explicit all-pairs alignment [1, 2]. The table itself, however, grows with $\boldsymbol{H \cdot L}$ and soon exceeds the memory of a single node. Distributing it across nodes removes that limit, but the result is only useful if it reproduces FastGA's alignments exactly.

Results: DistributedPanGA partitions FastGA's index, seed generation and alignment across MPI ranks while keeping FastGA's alignment code. On yeast and primate pangenomes of 83 Mbp to 2.63 Gbp, every run is byte-identical to FastGA and, at equal core counts, 1.0–1.9× faster. On 90 human haplotypes of chromosome 6 (15.5 Gbp), four nodes finish in 180 s against FastGA's best one-node time of 1,480 s; output is identical at 32 ranks, while at 64 ranks one of 1.38 million alignments differs and is under investigation. Alignment dominates the run time, and its uneven cost across ranks limits further scaling.

Availability and Implementation: DistributedPanGA is implemented in C with MPI and is available at https://github.com/SakshamDiwan/DistributedPanGA. Benchmark pangenomes are from Zenodo [3]. [TODO: license and archived release]

Contact: egarris5@uthsc.edu, gg434@cornell.edu

Supplementary Information: Supplementary data are available at *Bioinformatics* online.

1 Introduction

1.1 Motivation

FastGA [1] aligns genome assemblies on one shared-memory node. To self-align a pangenome, all haplotypes go into one index and are aligned against themselves, so one scan of the index finds matches across every pair of haplotypes [1, 2]. The index grows with the total sequence length and soon exceeds the memory of a single node.

1.2 Problem

We distribute FastGA's self-alignment across nodes. The result is only useful if it reproduces FastGA's alignments exactly, so we require byte-identical output.

1.3 Prior work

HySortK [4] counts k-mers in distributed memory by sorting. ELBA [5] generates contigs for long-read assembly in distributed memory. [TODO: other prior work, with verified citations]

1.4 Contributions

- DistributedPanGA, an MPI implementation of FastGA's self-alignment, byte-identical to FastGA on every yeast and primate run and on human chromosome 6 at 32 ranks.
- All-to-all exchanges of more than 2^{31} bytes per rank, and seeds kept in memory from generation to alignment.
- An evaluation on three pangenomes of 83 Mbp to 2.63 Gbp: at equal core counts DistributedPanGA is 1.0–1.9× faster than FastGA, and on primates14.chr6 two nodes are 1.2× faster than FastGA's fastest one-node run.
- On 90 human haplotypes of chromosome 6 (15.5 Gbp), four nodes are 8.2× faster than FastGA's fastest one-node run.

2 System and methods

2.1 FastGA

FastGA [1] aligns genome assemblies on a single shared-memory node. It stores the input as a genome database (GDB) of 2-bit packed sequence and builds a genome index (GIX): the sorted 40-mers sampled at closed-syncmer positions (Section 3.1), with the longest common prefix (LCP) of each entry with its predecessor and every occurrence's contig, offset and strand. Seeds are adaptive seeds (adaptamers): for each occurrence, the longest prefix it shares with another indexed k-mer, kept only if fewer than f entries share it ($f = 10$ by default), and paired with every other occurrence of that prefix. Seeds are sorted by contig pair and diagonal, diagonal bands with sufficient seed coverage are extended into local alignments, and `ALNtoPAF` converts FastGA's `.1aln` output to PAF. For a pangenome, all haplotypes are placed in one GDB and aligned against themselves, so one index and one merge cover every pair of haplotypes [2].

For H haplotypes of length about L, indexing and the merge scan are $O(HL)$. So is seed generation, with at most $f - 1$ seeds per occurrence, but only because adaptamers occurring f or more times (e.g. a 40-mer conserved in f haplotypes) are discarded. Alignment has no such bound: when most haplotype pairs align end to end, the output alone grows quadratically in H.

2.2 DistributedPanGA

DistributedPanGA [6] computes the same self-alignment as FastGA with P single-threaded MPI ranks, which read one shared GDB on demand. The pipeline has six stages (Section 3).

1. **Extraction:** each rank emits one record per sampled k-mer occurrence in its range of whole contigs.
2. **Distributed sort:** records are exchanged by k-mer prefix and sorted locally; the slices, in rank order, hold FastGA's index entries in k-mer order.
3. **Seed generation:** each rank runs FastGA's adaptamer merge on its slice.
4. **Seed redistribution:** each seed goes to the single rank that owns its contig pair.
5. **Alignment:** each rank aligns its own pairs with FastGA's code.
6. **Output:** each rank writes PAF with `ALNtoPAF`; the union of the per-rank files is the result.

Syncmer selection, the adaptamer merge, alignment and PAF conversion are FastGA code, kept close to upstream. Ours are the extraction partition and in-window k-mer packing, the distributed sort, an adapter that presents a rank's slice through FastGA's index interface in place of a GIX file, the deterministic assignment of contig pairs to ranks, and the two bounded all-to-all exchanges (Section 3.2).

Our correctness criterion is exact agreement with an independent single-node FastGA run on the same GDB: both PAF files are sorted bytewise, keeping duplicates, and compared byte for byte. This multiset equality is stronger than matching alignment counts.

3 Algorithm

3.1 Seed extraction

Extraction reproduces FastGA's k-mer sampling exactly. Each rank scans a contiguous range of whole contigs, balanced by length, in windows of up to 10 Mbp. The start j of a 12-mer is selected when the 12-mer is a closed syncmer: the minimum of its five canonical 8-mers, under FastGA's fixed (non-lexicographic) order, occurs at its first or last 8-mer position. With ties, an occurrence at either end suffices.

Each selected position yields up to two records: the forward 40-mer starting at j and the reverse complement of the 40-mer ending at the last base of the syncmer; a strand is skipped where its 40-mer would leave the contig. A record stores the 2-bit packed k-mer with its contig, position and strand. Occurrences, not k-mer counts, are needed because each seed pairs two occurrences.

Every window after the first is extended 40 bases to the left, so that reverse-complement k-mers and forward k-mers starting before the window lie inside it. Only a forward k-mer that runs past the right edge of a non-final window is read separately from the GDB. The scan state carries across windows, so the selected positions and the order of emitted records do not depend on the window size.

3.2 Data movement between ranks

Ranks exchange data twice: records during the distributed sort, and seeds before alignment.

Distributed sort.

Each record is assigned to one of 1,024 buckets by the first five bases of its k-mer. One all-reduce of bucket counts lets every rank compute the same split into P contiguous bucket ranges of about $1/P$ of the records, by the rule FastGA uses to divide its index among threads. Each rank then radix-sorts its slice and recomputes LCPs, correcting the first against the preceding rank's last k-mer. Because buckets are never split, seed generation needs no communication: FastGA's merge pairs only occurrences sharing their first 12 bases, so every candidate group, including the full count tested against the cutoff, lies on one rank, and each rank produces exactly its share of FastGA's seeds.

Seed redistribution.

Each seed is sent to the rank that owns its contig pair. Every rank computes the same pair-to-rank map from contig lengths alone, giving pairs in decreasing order of length product to the rank with the smallest accumulated product, so no communication is needed. The merge writes each seed directly into an in-memory buffer for its destination rank.

Bounded exchanges.

Both exchanges are personalized all-to-all transfers of fixed-size records whose volume per rank can exceed the 32-bit counts of `MPI_Alltoallv`. Large transfers are split into rounds of whole records, and the result is byte-identical to a single unbounded exchange for any number of rounds (Section 4.2). The per-round budget bounds only staging memory: the complete send and receive data stay resident (during the sort, a rank holds its extracted records, their packed send copy and its received slice), so peak memory per rank is set by its volume of records and seeds, not by the budget (Section 4).

3.3 Distributed alignment

Each rank aligns the contig pairs it owns (Section 3.2) with FastGA's code. The map covers all $N(N+1)/2$ pairs of the N contigs, including each contig with itself; ties are broken by contig index, so every rank computes the same map. Both orientations of a pair map to the same rank. Seeds are split into two sets: same strand and opposite strands.

Each rank groups its seeds by contig pair, sorts them by diagonal, and aligns each pair and strand set with FastGA's routine, which chains seeds, extends chains into local alignments and removes redundant ones. Each rank writes its own `.1aln` file and converts it to PAF with `ALNtoPAF`; the output is the union of these files.

A pair's alignments depend only on its seeds and its two contigs, and all its seeds reach one rank, so each rank produces exactly FastGA's alignments for its pairs. The map balances contig length products, not alignment cost.

4 Implementation

4.1 Extraction I/O

Per-rank timers showed that extraction dominated early runs and barely shortened as ranks were added. Although the scan held its window in memory, each selected k-mer was fetched again from the GDB. The bytes came from the file buffer, but every fetch repositioned the file: system-call tracing counted one `lseek` per record. Kernel time per rank stayed roughly constant as work per rank fell. We did not isolate why (shared-file-system contention is one candidate); the change removes the calls regardless.

Each k-mer is now packed from the window buffer, extended by 40 bases of left context (Section 3.1); only forward k-mers crossing a window's right edge are still fetched. Both paths share one packing routine and the selection loop is unchanged, so only the source of the bases changes, not which records are emitted or in what order.

Validation has three levels. Extraction output is byte-identical to the previous implementation and to a record dump from an instrumented copy of FastGA's `GIXmake`. Unit tests with a 64-base window compare against a single-window run, check that both paths fire and together account for every record, and run under address and undefined-behaviour sanitizers. End-to-end runs match the FastGA reference (Section 2.2).

4.2 Large-count exchanges

Both exchanges originally used one `MPI_Alltoallv`, whose 32-bit counts made a rank abort above $2^{31}-1$ bytes; primates4.chr6 could not run at small rank counts.

Both now use one routine. Ranks exchange per-peer sizes as 64-bit integers and agree on the smallest per-round budget (1 GiB by default). If every rank's totals fit, a single `MPI_Alltoallv` is used as before. Otherwise each peer's data moves in rounds of at most c bytes: the budget divided by P, rounded down to whole records and capped so displacements fit in an `int`. All ranks run the same number of rounds, and each received chunk is copied to its final offset, so the output is identical to one exchange and staging stays below twice the budget. Seed redistribution supplies data through a callback that reads the per-destination seed buffers (Section 4.3) directly. The planning arithmetic has no MPI calls, so unit tests exercise sizes far above 2^{31} without allocating them.

Unit and MPI tests on 1–8 ranks with budgets down to 256 bytes pass, and with an 8 MiB budget, which forces rounds in both exchanges, per-rank PAF files on yeast7 and scerevisiae8 are byte-identical to the single-exchange version.

4.3 In-memory seeds

Seed generation used to write each destination's seeds to a scratch file. The exchange read these files and wrote the received seeds to another file, which was read twice more: to count seeds per contig and to load them for alignment.

Seeds now go into one growing buffer per destination rank. The exchange copies them into one receive buffer and frees each send buffer after sending; counting and loading read the receive buffer, which is freed after loading. On one rank, no copy is made. Alignment files (`.las`, `.1aln`) still go to scratch.

The output is unchanged and seeds no longer touch files. The cost is memory: a rank holds every seed it sends and receives, plus the exchange's staging buffers, which the round budget bounds.

5 Results

5.1 Experimental setup

We ran DistributedPanGA and FastGA on the three pangenomes in Table 1 and compared their output and their running time. DistributedPanGA ran on one to four nodes; FastGA, which uses threads on a single machine, ran on one node. All runs used dedicated CPU nodes on Perlmutter, each with 128 cores and 512 GB of memory. Both tools used FastGA's default parameters.

A run with P ranks on n nodes places P/n ranks on each node and pins each rank to an equal share of the node's cores. A rank is single-threaded except during the local k-mer sort, which uses four threads.

We report the time of each of four stages and their sum, the total time. The stages are extraction; the distributed sort; seed generation and redistribution (Section 2.2, stages 3–4); and alignment with PAF output (stages 5–6). Ranks that finish extraction early wait at the start of the sort for the slowest

one; we count that wait as extraction, not as sort. Each configuration ran three times. We report the run with the median total time, so its stage times add up to its total.

FastGA ran on one node with t threads. Its time is the sum of `GIXmake`, `FastGA` and `ALNtoPAF`. `GIXmake` accepts at most 32 threads, so for $t > 32$ it ran with 32. `FastGA` crashes at startup with 125 or more threads on all three datasets, so we measured it up to $t = 64$. `FAtoGDB` is excluded because both tools start from the same GDB.

A run is correct if the concatenation of its per-rank PAF files, sorted, is byte-identical to FastGA's sorted PAF for the same input.

Table 1 Datasets. yeast7 is seven *S. cerevisiae* strains; the primate and human sets are chromosome 6 assemblies from the PGGB benchmark collection [3]. Contigs are the GDB's gap-free sequences.

Dataset	Bases	Sequences	Contigs	FastGA alignments
yeast7	83.4 Mbp	112	119	43,126
primates4.chr6	672.5 Mbp	4	100	1,547,485
primates14.chr6	2.63 Gbp	52	82	4,577,556
hsapiens90.chr6	15.51 Gbp	1,410	1,422	1,375,248

5.2 Yeast and primate datasets

All 30 DistributedPanGA runs produced output byte-identical to FastGA's. Table 2 gives the time of each stage and the total for every configuration, and Table 3 the same for FastGA.

Table 2 DistributedPanGA time per stage, in seconds. "Seeds" is seed generation and redistribution; "Alignment" includes PAF output. "Total" is the sum of the four stages. Each row is the run with the median total time of three.

Dataset	Nodes	Ranks	Extraction	Sort	Seeds	Alignment	Total
yeast7	1	1	1.63	2.55	9.87	65.92	79.97
yeast7	1	2	0.86	1.38	7.51	33.05	42.81
yeast7	1	4	0.48	0.79	3.88	17.49	22.64
yeast7	1	8	0.24	0.42	2.09	9.95	12.69
primates4.chr6	1	4	3.14	7.43	16.62	184.29	211.47
primates4.chr6	1	8	2.90	4.60	9.23	97.29	114.02
primates4.chr6	1	16	2.50	3.01	5.22	70.03	80.76
primates14.chr6	1	16	6.10	9.76	19.50	1,428.58	1,463.94
primates14.chr6	2	32	4.65	6.73	11.07	1,196.09	1,218.54
primates14.chr6	4	64	3.65	5.00	7.53	1,212.91	1,229.09

Strong scaling on one node.

From 1 to 8 ranks, yeast7 drops from 79.97 s to 12.69 s, a speedup of 6.3 (79% parallel efficiency; Fig. 1a). From 4 to 16 ranks, primates4.chr6 drops from 211.47 s to 80.76 s, a speedup of 2.6 (65%). Alignment takes 77–87% of the total time in these runs.

Strong scaling across nodes.

With 16 ranks per node, primates14.chr6 takes 1,463.94 s on one node, 1,218.54 s on two and 1,229.09 s on four. Alignment takes 98–99% of the total time and stays between 1,196.09 s and 1,428.58 s (Fig. 1d). From 16 to 64 ranks, extraction falls from 6.10 s to 3.65 s, the sort from 9.76 s to 5.00 s, and seed generation and redistribution from 19.50 s to 7.53 s.

Table 4 breaks the alignment stage down by rank. From 16 to 64 ranks on primates14.chr6, the average rank's alignment time falls from 722.15 s to 183.43 s, while the slowest rank's stays between 1,169.50 s and 1,393.88 s. At 64 ranks the slowest rank takes 6.59 times the average; on primates4.chr6 the ratio grows from 1.09 at 4 ranks to 1.70 at 16.

Table 3 FastGA time per step on one node, in seconds. Each row is the run with the median total time of three. `GIXmake` builds the index, `FastGA` aligns and `ALNtoPAF` writes PAF.

Dataset	Threads	`GIXmake`	`FastGA`	`ALNtoPAF`	Total
yeast7	1	9.05	72.32	0.20	81.57
yeast7	2	5.59	42.09	0.12	47.80
yeast7	4	3.32	24.29	0.08	27.69
yeast7	8	2.42	15.07	0.06	17.55
primates4.chr6	4	23.95	272.65	1.51	298.12
primates4.chr6	8	19.62	171.21	1.78	192.61
primates4.chr6	16	18.94	129.69	0.79	149.42
primates14.chr6	16	67.50	2,611.25	6.01	2,684.77
primates14.chr6	32	83.18	1,452.68	5.33	1,541.20
primates14.chr6	64	79.96	1,401.42	5.15	1,486.52

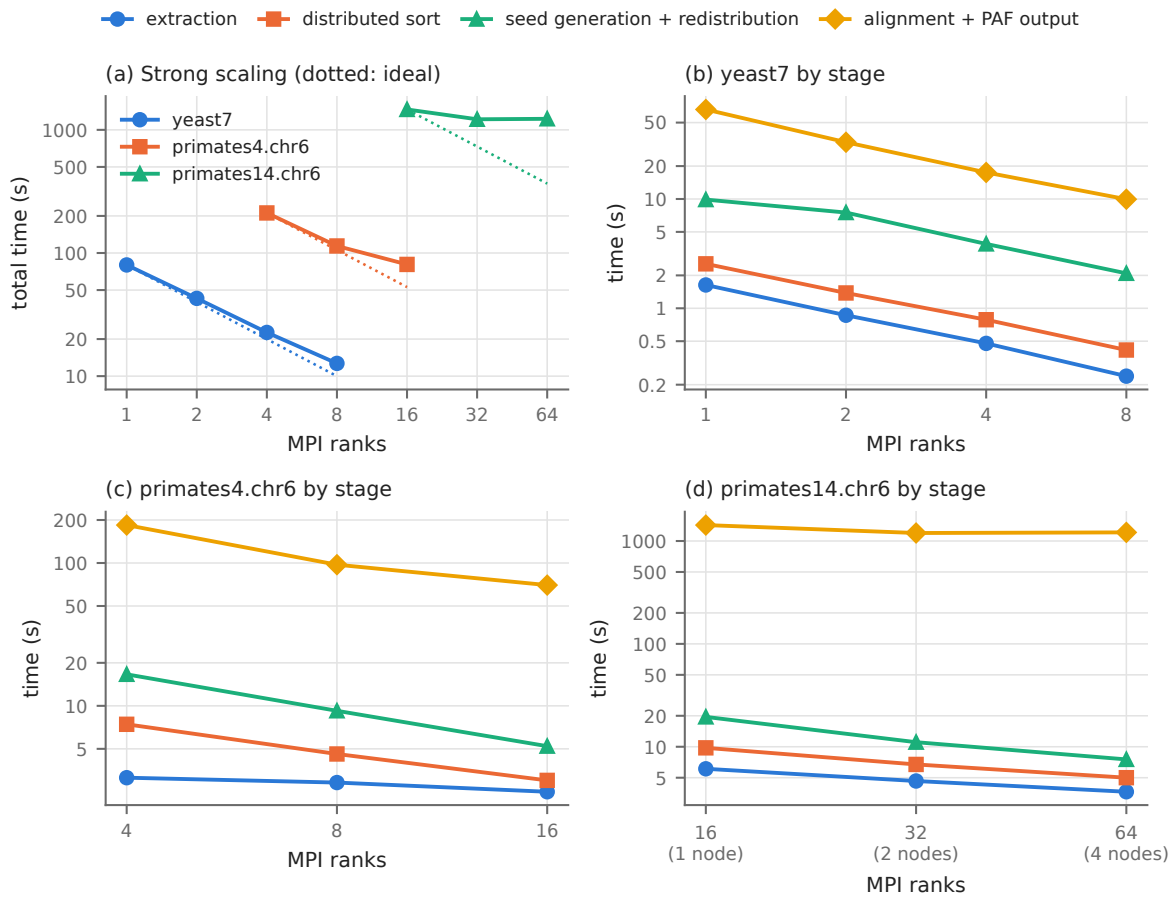


Fig. 1 (a) Total time against ranks. yeast7 and primates4.chr6 run on one node. primates14.chr6 uses 16 ranks per node (1, 2 and 4 nodes). Dotted lines show ideal scaling from the smallest run. (b)–(d) Time per stage for each dataset; the legend at the top applies to these panels.

Comparison with FastGA.

All FastGA runs produced the same alignments as the reference. At equal core counts, DistributedPanGA is 1.0–1.4× faster than FastGA on yeast7 (1 to 8 cores), 1.4–1.9× on primates4.chr6 (4 to 16 cores) and 1.2–1.8× on primates14.chr6 (16 to 64 cores) (Fig. 2). From 4 to 16 cores on primates4.chr6, FastGA speeds up by 2.0 and DistributedPanGA by 2.6. From 16 to 64 cores on primates14.chr6, FastGA speeds up by 1.8 and DistributedPanGA by 1.2.

Table 4 Alignment time per rank, in seconds, for the runs in Table 2: the slowest, average and fastest rank, and the slowest divided by the average. Alignment time here is the time spent aligning seed chains, without sorting seeds or writing PAF.

Dataset	Nodes	Ranks	Slowest	Average	Fastest	Slowest/avg.
primates4.chr6	1	4	159.62	146.71	128.52	1.09
primates4.chr6	1	8	86.10	70.14	54.74	1.23
primates4.chr6	1	16	63.38	37.34	27.90	1.70
primates14.chr6	1	16	1,393.88	722.15	229.84	1.93
primates14.chr6	2	32	1,169.50	362.12	55.76	3.23
primates14.chr6	4	64	1,209.48	183.43	32.11	6.59

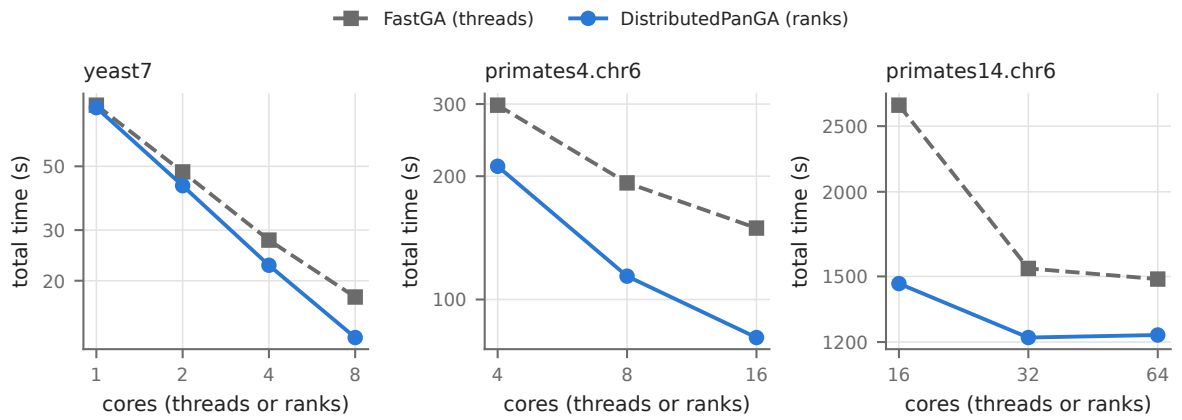


Fig. 2 DistributedPanGA (P ranks) against FastGA (t threads on one node) at equal core counts. For primates14.chr6, DistributedPanGA uses 16 ranks per node.

FastGA's fastest one-node time on primates14.chr6 is 1,486.52 s, at 64 threads. DistributedPanGA takes 1,463.94 s on one node, 1,218.54 s on two and 1,229.09 s on four, 1.0, 1.2 and 1.2× faster than that time.

Weak scaling.

Two sets of three runs have about the same number of bases per rank: 83 Mbp (yeast7 on 1 rank, primates4.chr6 on 8, primates14.chr6 on 32 over two nodes) and 42 Mbp (2, 16 and 64 ranks). At 83 Mbp per rank, seed generation and redistribution take 9.87, 9.23 and 11.07 s; extraction grows 2.9× (1.63 to 4.65 s), the sort 2.6× (2.55 to 6.73 s) and alignment 18× (65.92 to 1,196.09 s).

Table 5 gives the alignment output per rank for these runs. At 83 Mbp per rank, from yeast7 to primates14.chr6 the aligned bases per rank grow 7.9×, the average rank's alignment time 8.6× and the slowest rank's 28×. primates4.chr6 aligns 1.07× as many bases per rank as yeast7, in 4.5× as many alignments, and its average rank takes 1.7× as long.

5.3 Human chromosome 6

hsapiens90.chr6 holds 90 human haplotypes of chromosome 6 from the same collection [3]: 15.51 Gbp in 1,410 sequences and 1,422 contigs, six times the size of primates14.chr6. FastGA's reference has 1,375,248 alignments and links every one of the 4,005 pairs of haplotypes. We ran DistributedPanGA with 32 and 64 ranks on four nodes and FastGA on one node with 16 and 32 threads; each configuration ran twice for DistributedPanGA (totals within 1%) and once for FastGA. `GIXmake` crashes with 32 threads on this input, so FastGA's index was built with 16 threads in both FastGA rows.

Correctness.

With 32 ranks, both runs are byte-identical to FastGA's sorted output. FastGA's output is the same at 16 and 32 threads. With 64 ranks, both runs miss the same single alignment, a same-strand self-alignment of one contig over about 3.9 kbp; every other line matches. The seeds for that contig pair are identical

Table 5 Weak scaling: alignment output and alignment time per rank. Aligned bases are the summed lengths of all alignments in FastGA's output, which every run reproduces, divided by the ranks; one input base is counted once for every alignment it is part of. Alignment time is as in Table 4.

		Input	Aligned	Alignments	Alignment time (s)	
Dataset	Ranks	Mbp/rank	Mbp/rank	per rank	Average	Slowest
yeast7	1	83.4	545.0	43,126	41.91	41.91
primates4.chr6	8	84.1	584.3	193,436	70.14	86.10
primates14.chr6	32	82.2	4,309.4	143,049	362.12	1,169.50
yeast7	2	41.7	272.5	21,563	20.20	20.26
primates4.chr6	16	42.0	292.2	96,718	37.34	63.38
primates14.chr6	64	41.1	2,154.7	71,524	183.43	1,209.48

at 32 and 64 ranks, so the difference arises during alignment. We have not yet determined whether it comes from our pipeline or from an order-dependent edge case in FastGA's alignment code.

Time and memory.

Table 6 and Fig. 3 give the times. DistributedPanGA takes 291.17 s with 32 ranks and 180.07 s with 64, a speedup of 1.62. FastGA takes 1,480.42 s with 16 threads and 1,736.93 s with 32, so DistributedPanGA with 64 ranks is 8.2× faster than FastGA's fastest run, and 6.0× faster at 32 ranks against 32 threads. Alignment is 84–87% of DistributedPanGA's total. The slowest rank's alignment time is 1.32 times the average at 32 ranks and 1.49 at 64. FastGA peaks at 15.1 GB on one node. DistributedPanGA peaks at 22.9 GB per rank with 32 ranks and 14.1 GB with 64; summed over ranks, that is 675 GB and 747 GB.

Table 6 hsapiens90.chr6, time per stage in seconds. DistributedPanGA ran on four nodes; stages as in Table 2. For FastGA (one node), "Index" is `GIXmake` (16 threads in both rows), "Align" is `FastGA` and "PAF" is `ALNtoPAF`. Peak memory is per rank for DistributedPanGA and per process for FastGA.

DistributedPanGA	Ranks	Extraction	Sort	Seeds	Alignment	Total	Peak GB
	32	9.86	22.65	6.46	252.20	291.17	22.9
	64	6.13	18.16	3.85	151.92	180.07	14.1
FastGA	Threads	Index	Align	PAF		Total	Peak GB
	16	400.39	1,069.81	10.22		1,480.42	13.4
	32	400.39	1,330.71	5.83		1,736.93	15.1

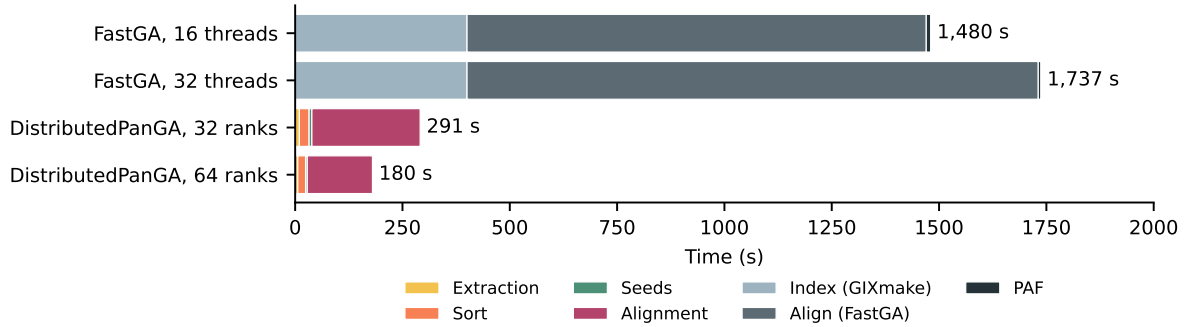


Fig. 3 hsapiens90.chr6: time per stage for FastGA on one node and DistributedPanGA on four nodes (Table 6).
Alt text: Horizontal stacked bars. FastGA takes about 1,480 and 1,737 seconds, mostly alignment plus a 400-second index build; DistributedPanGA takes about 291 and 180 seconds, mostly alignment.

5.4 Scaling across datasets

Fig. 4 compares the four datasets. At the largest doubling of ranks tested for each, the total time falls by 1.78× on yeast7 (4 to 8 ranks), 1.41× on primates4.chr6 (8 to 16), 0.99× on primates14.chr6 (32 to 64) and 1.62× on hsapiens90.chr6 (32 to 64). At its largest rank count, the slowest rank's alignment time is 1.70 times the average on primates4.chr6, 6.59 on primates14.chr6 and 1.49 on hsapiens90.chr6.

On hsapiens90.chr6, the per-pair records sum to 6,013 s of alignment work at 32 ranks and 6,426 s at 64, and the most expensive contig pair, a self-alignment, takes 34 s, about 0.5% of the total. On primates14.chr6 the most expensive pair takes 773 s, about 7% of all alignment work (measured with an earlier build that runs the same alignment code). From 32 to 64 ranks on hsapiens90.chr6, the alignment stage falls from 252.20 s to 151.92 s, while the average rank's alignment time falls from 187.12 s to 100.47 s; the sort falls from 22.65 s to 18.16 s. Peak memory per rank falls from 22.9 GB to 14.1 GB, and memory summed over ranks rises from 675 GB to 747 GB.

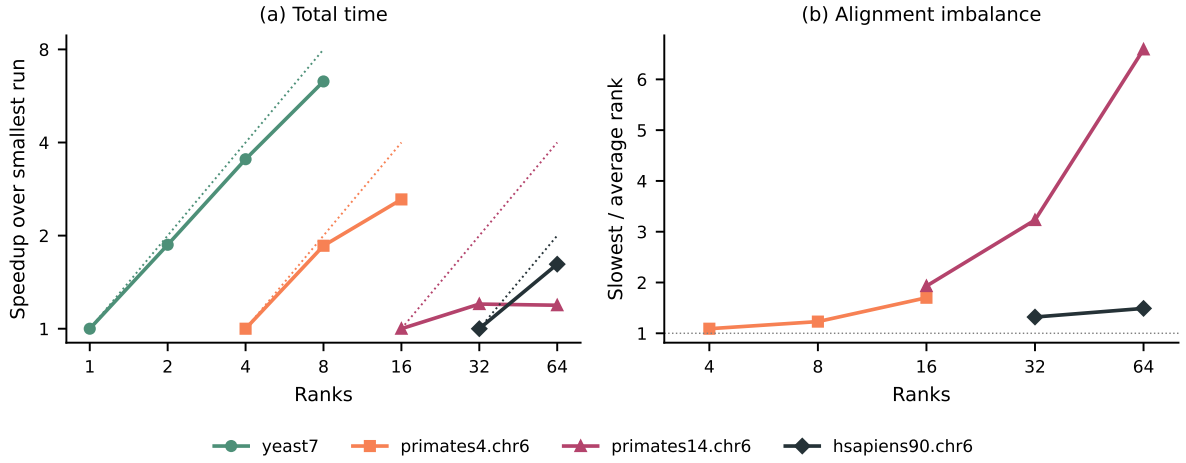


Fig. 4 Scaling across the four datasets. (a) Speedup of the total time over each dataset's smallest run; dotted lines show ideal scaling. (b) Slowest rank's alignment time divided by the average (Table 4 and Section 5.3).

Alt text: Two line plots against ranks. Yeast7 and the human dataset stay close to ideal speedup, primates4.chr6 falls below it, and primates14.chr6 stops improving after 32 ranks. Alignment imbalance rises to 6.6 on primates14.chr6 at 64 ranks but stays below 1.5 on the human dataset.

6 Discussion

6.1 Small-dataset results

Alignment limits scaling.

On primates14.chr6 every stage except alignment keeps getting faster up to 64 ranks, while alignment stops improving at about 1,200 s. Table 4 points to how the work is assigned. The average rank's alignment time halves each time the ranks double, as it should, but the slowest rank's barely changes, and the alignment stage lasts as long as that slowest rank. The work is therefore divisible in total but unevenly assigned: at 64 ranks the fastest rank finishes in 32 s and waits about 20 minutes for the slowest. We think the cause is the unit of assignment. A rank aligns all seeds of the contig pairs it owns, and a pair is never split, so one expensive pair sets a lower bound on the alignment stage.

primates4.chr6 at 16 ranks.

Efficiency drops from 93% at 8 ranks to 65% at 16. The same pattern appears on a smaller scale: the slowest rank's alignment time grows from 1.23 to 1.70 times the average (Table 4).

Extraction is divided the same way.

Extraction also assigns whole contigs to ranks, so it becomes uneven once there are more ranks than large contigs. On primates4.chr6 each rank extracts 166–170 Mbp at 4 ranks but 14–153 Mbp at 8; on

primates14.chr6 at 64 ranks, 36 of the 64 ranks extract nothing. Extraction takes only seconds here, so the cost is small, but it is the same limit as in alignment: work that comes in units too large to split evenly.

Weak scaling.

At a fixed number of bases per rank, seed generation and redistribution stay flat. Extraction grows because whole contigs cannot be divided evenly across more ranks, and the sort grows because its all-to-all exchange takes longer with more ranks. Alignment grows 18× from yeast7 to primates14.chr6, for two reasons (Table 5).

First, equal input does not mean equal alignment work. The work depends on how much of the input matches other input: a stretch of one assembly that matches 13 others is aligned 13 times. The 14 primate assemblies match each other almost end to end, so each rank aligns 7.9× as many bases as on yeast7, and the average rank's alignment time grows by a similar 8.6×. The number of alignments matters too: primates4.chr6 aligns about as many bases per rank as yeast7 but in 4.5× as many alignments, and takes 1.7× as long.

Second, the stage lasts as long as its slowest rank, and the work is unevenly assigned (Section 6.1, first paragraph). On primates14.chr6 the slowest rank takes 3.2× the average, which together with the first factor accounts for its 28× growth.

Against FastGA.

DistributedPanGA is faster than FastGA at every equal core count tested. Both tools run FastGA's alignment code, so the difference comes from how the work is split. On yeast7 and primates4.chr6 the gap widens with cores. On primates14.chr6 it narrows, from 1.8× at 16 cores to 1.2× at 64, because both tools stop improving after 32 cores: from 32 to 64, FastGA speeds up by 1.04× and DistributedPanGA not at all. This fits a limit both share: FastGA, like DistributedPanGA, aligns each contig pair on a single thread, so the most expensive pair bounds both.

On one node, DistributedPanGA with 16 ranks matches FastGA's fastest time (64 threads); on two nodes it is 1.2× faster. Four nodes add nothing, for the reason above: until the work is assigned more evenly, more nodes do not shorten the slowest rank.

6.2 Scaling across datasets

The human dataset, six times larger than primates14.chr6, scales much better, which fits the explanation in Section 6.1: a contig pair is never split, so the most expensive pair bounds the alignment stage. hsapiens90.chr6 has 1,422 contigs against 82, and its most expensive pair is about 0.5% of the alignment work rather than 7%. Its work could in principle be spread over roughly 180 ranks before that pair sets the stage time, against about 14 for primates14.chr6. How many contigs an input has, and how evenly its work falls on them, may therefore matter more for scaling than its size.

The task bound does not explain the remaining imbalance on the human data. At 64 ranks its slowest rank aligns for 149 s, more than four times its most expensive pair (34 s) and 1.49 times the average (100 s). Since no single task is that large, the gap most likely comes from the static assignment by length product, which does not track alignment cost (Section 6.4). A perfectly balanced assignment would shorten the alignment stage by at most this factor of about 1.5 at 64 ranks.

Memory per rank falls as ranks are added, but less than in proportion, so the total grows. Per-rank memory therefore decides how few nodes a run can use: 32 ranks at 22.9 GB each should fit on two of Perlmutter's 512 GB nodes, where we used four; we have not tested this.

6.3 Human scale

FastGA handled hsapiens90.chr6 on a single node in under half an hour with 15 GB of memory, so at the scale of one chromosome, distribution shortens the run rather than making it possible. The input is six times larger than primates14.chr6, yet FastGA's best time is about the same on both, and DistributedPanGA is several times faster on it. A likely reason is that the 90 haplotypes are nearly identical: most k-mers should then occur at least $f = 10$ times and yield no seeds, leaving less alignment work. We did not measure this directly. Where single-node FastGA reaches its limits, in more haplotypes or whole genomes, remains to be measured.

DistributedPanGA's speed comes with memory: summed over ranks it used about 45–50 times FastGA's peak. Each rank holds its records and seeds in memory, and ranks are single-threaded, so more ranks, not threads, are the way to use a node's cores.

The single alignment missing at 64 ranks is the only disagreement with FastGA across all datasets and rank counts. Because the seeds agree, it arises in the alignment stage; whether it lies in our pipeline or in an order-dependent edge case of FastGA's alignment code is under investigation. Until it is explained, exact agreement is established for human chromosome 6 at 32 ranks only.

6.4 Limitations and future work

Workload-aware load balancing.

Contig pairs could be weighted by estimated work rather than length, as in the multiway number partitioning of ELBA [5], provided every rank still derives the assignment without communication. However, pre-alignment features such as seed counts only partly predict alignment time in our measurements. Replaying measured task times under a dynamic schedule (self-alignments first, by contig length) shortens the slowest rank in simulation only; it is not implemented, and it conflicts with sending seeds to their owner before alignment. No schedule can finish before the largest single task (Section 6.1); that requires splitting self-alignments across ranks while reproducing FastGA's alignments exactly, an open problem.

GPU acceleration.

Syncmer selection and radix sorting are regular, data-parallel passes over packed 2-bit sequence and candidates for GPU offload, but they now take a small share of run time; alignment, which dominates, is irregular and harder to offload.

Acknowledgements. [TODO: acknowledgements]

Author contributions. S.D. and H.C. developed the software, ran the experiments and wrote the manuscript; S.D. led the in-memory seed exchange, the distributed alignment and the yeast and primate evaluation, and H.C. the extraction I/O, the large-count exchanges and the human evaluation. S.C., E.G. and G.G. conceived and supervised the project. [TODO: confirm roles]

Supplementary data. Supplementary data are available at *Bioinformatics* online.

Conflict of interest. None declared.

Funding. [TODO: funding sources and grant numbers]

Data availability. All pangenomes are public on Zenodo [3]. The job scripts behind every result and the checksums of the inputs and FastGA reference outputs are in the repository's `jobs/` directory.

References

[1] Myers, G., Durbin, R. & Zhou, C. FastGA: fast genome alignment. *Bioinformatics Advances* **5**, vbaf238 (2025).

[2] Garrison, E. FastGA for pangenome self-alignment. Unpublished workflow document (2025).

[3] Guarracino, A. Pangenomes of multiple species for the "building pangenome graphs" publication. Zenodo (2023).

[4] Li, Y. & Guidi, G. *High-performance sorting-based k-mer counting in distributed memory with flexible hybrid parallelism. Proceedings of the 53rd International Conference on Parallel Processing (ICPP '24)*, 919–928 (ACM, 2024).

[5] Guidi, G. *et al. Distributed-memory parallel contig generation for de novo long-read genome assembly. Proceedings of the 51st International Conference on Parallel Processing (ICPP '22)*, 1–11 (ACM, 2022).

[6] Diwan, S. DistributedPanGA: A distributed-memory pangenome self-alignment pipeline. https://github.com/SakshamDiwan/DistributedPanGA (2026).